

Frontend Architecture & Engineering Standards

Comprehensive Master Guide: Project Structures, Best Practices & Technical Conventions

1. Production-Ready React & Redux Toolkit Architecture

Feature-Based Project Directory Structure

```
src/
├── assets/                # Static assets (images, icons, fonts, global styles)
├── components/           # Shared presentation components (Button, Modal, Input)
│   └── ui/               # Atomic UI elements / Design system primitives
├── config/               # Global app config (env variables, constants)
├── context/              # React Context providers (Theme, Auth state)
├── features/             # Feature modules (Domain-driven architecture)
│   ├── auth/            # Feature: Authentication
│   │   ├── api/         # Feature-specific endpoints / RTK Query slices
│   │   ├── components/  # Feature-specific components (LoginForm, SignupCard)
│   │   ├── hooks/       # Feature-specific custom hooks
│   │   ├── slice/       # Redux Toolkit state slice (authSlice.ts)
│   │   └── types/       # Feature domain TypeScript definitions
│   ├── dashboard/      # Feature: Dashboard Analytics
│   ├── hooks/          # Shared custom React hooks (useDebounce, useLocalStorage)
│   ├── layouts/        # Page structure templates (SidebarLayout, AuthLayout)
│   ├── routes/         # Router configuration, protected routes & code splitting
│   ├── services/       # Global API instances (Axios client, WebSocket setup)
│   ├── store/          # Redux Toolkit store setup & RootState/AppDispatch exports
│   │   ├── index.ts    # Store configuration (`configureStore`)
│   │   └── rootReducer.ts # Combined reducers
│   ├── styles/         # Global CSS/SCSS, Tailwind config, design tokens
│   ├── types/          # Global TypeScript interfaces and utility types
│   ├── utils/          # Pure helper functions (formatters, validators, calculations)
│   ├── App.tsx         # Root application component
│   └── main.tsx        # Application entry point (DOM render)
```

2. React & Redux Toolkit (RTK) Best Practices

- **React Functional Components & Hooks:** Prefer functional components with custom hooks. Keep components clean and focused purely on UI presentation.
- **React Component Separation:** Separate "Smart" Container components (data fetching, state management) from "Dumb" Presentational components (pure UI rendering via props).
- **React Performance Optimization:** Use `React.memo` for heavy list items, and wrap callback functions in `useCallback` or expensive calculations in `useMemo` to prevent re-renders.
- **Redux Use Redux Toolkit (RTK):** Always use RTK standard modern APIs (`createSlice`, `createAsyncThunk`, or RTK Query) instead of legacy Redux boilerplate.
- **Redux Typed Hooks:** Export custom typed hooks `useAppDispatch` and `useAppSelector` derived from `RootState` and `AppDispatch` to ensure full type safety across components.

- **Redux Normalized State:** Store normalized data entities using RTK's `createEntityAdapter` to avoid deeply nested state mutation bugs and simplify lookups.

3. JavaScript (ES6+) & TypeScript Best Practices

- **TypeScript Strict Mode Enabled:** Ensure `"strict": true` is set in `tsconfig.json`. Avoid `any` at all costs; use `unknown` when the type is unknown, followed by type narrowing.
- **TypeScript Interfaces vs Types:** Use `interface` for object shapes and component props (enables declaration merging), and `type` for unions, primitives, or tuple definitions.
- **JavaScript Immutability & Declarative Code:** Avoid mutating variables. Use `const` by default, `let` only when reassigning. Use array methods (`map` , `filter` , `reduce`) over imperative `for` loops.
- **JavaScript Async Handling:** Prefer `async/await` over raw Promises for better readability. Wrap API calls in structured `try/catch` blocks with custom error handlers.
- **JavaScript Nullish Coalescing & Optional Chaining:** Safely navigate optional properties using `?.` and set fallbacks using `??` instead of loose truthy checks (`||`).

4. Modern HTML5 & CSS3 Standards

- **HTML Semantic Tags & Accessibility (a11y):** Use semantic tags (`<button>` , `<nav>` , `<article>`) and include proper ARIA attributes (`aria-label` , `aria-expanded`) for screen readers.
- **HTML Form Handling & Validation:** Always wrap inputs in a `<form>` tag with explicit submit handlers, proper `name` attributes, and accessible `<label>` bindings.
- **CSS Box Sizing & Reset:** Start all stylesheets with global `box-sizing: border-box` and clean resets to ensure cross-browser consistency.
- **CSS Architecture (BEM / CSS Modules):** Use CSS Modules or BEM (Block-Element-Modifier) naming strategies to avoid global selector collisions.
- **CSS Responsive Design:** Build mobile-first using relative CSS units (`rem` , `em` , `%`) and standard media query breakpoints. Avoid hardcoding pixel widths on container elements.